TASK 1 – FILE-PROCESSING DESIGN REVIEW (6 MARKS)

Task 1.1 – Diagnose the script (2 marks)

Deficiency	What could go wrong in the hostel office?
Hard-coded file path	The script works only on one computer and fails if the file is moved or the office user has a different folder.
File is not closed	The operating-system file handle remains open until cleanup. Repeated runs can waste resources or prevent other programs fro
Reads the whole file with read()	All 50,000 lines are loaded into memory together. A large file can use unnecessary memory.
Bare except with pass	It hides every error, including programming mistakes. The office cannot know why a record was skipped.
Uses int() for amount	A valid amount such as 1250.50 raises ValueError, so paise values are incorrectly rejected.
Repeated string concatenation	Repeated report = report + ... creates new strings and can be inefficient for a large report.
Blank final line	An empty line has no third field. It causes an error and is silently ignored.
No rejected-line record	Incorrect bills disappear without an audit trail. Staff cannot correct damaged records.
No validation of field count	A short line or a line with extra commas may be processed incorrectly or fail.

Task 1.2 – Repaired design (3 marks)

```
from decimal import Decimal, InvalidOperation

def process_mess_file(filename):
    total = Decimal("0.00")
    report_lines = []
    rejected_lines = []

    try:
        with open(filename, "r", encoding="utf-8") as file:
            for line_number, line in enumerate(file, start=1):
                line = line.strip()
                if not line:                # ignore blank lines
                    continue

                parts = line.split(",")
                if len(parts) != 3:
                    rejected_lines.append((line_number, line,
                                           "expected 3 fields"))
                    continue

                register_no, name, amount_text = parts
                try:
                    amount = Decimal(amount_text.strip())
                except InvalidOperation:
                    rejected_lines.append((line_number, line,
                                           "invalid amount"))
                    continue

                if amount < 0:
                    rejected_lines.append((line_number, line,
                                           "negative amount"))
                    continue

                total += amount
                report_lines.append(
                    f"{name.strip()} {amount:.2f}"
                )

    except FileNotFoundError:
        print(f"File not found: {filename}")
        return
    except PermissionError:
        print(f"Permission denied: {filename}")
        return
    except UnicodeDecodeError:
        print(f"File is not valid UTF-8: {filename}")
        return

    print("\n".join(report_lines))
    print("Total", f"{total:.2f}")
    print("Rejected lines:")
    for item in rejected_lines:
```

```
        print(item)

if __name__ == "__main__":
    process_mess_file("mess_oct.txt")
```

Decimal is used instead of float because it represents decimal currency values more accurately. The function accepts the file name as a parameter, so the same function can process different monthly files.

Task 1.3 – Design defence (1 mark)

The repaired program is safer for 50,000 lines because it reads one line at a time and does not keep the complete file in memory. The with statement closes the file automatically, even if an error occurs. Specific exceptions make file problems visible. Invalid records are saved with their line numbers instead of being silently discarded. Decimal correctly handles paise. A list stores report rows efficiently and join() creates the report once.

Alternative concept and complexity analysis

Using read() loads the entire file as one string. Using readlines() loads a list containing all lines. Both approaches still take O(n) time to read and process n characters/lines. Their extra memory is O(file size), because the complete content is retained. Line-by-line iteration also takes O(n) time, but its file-reading buffer is approximately O(one line) rather than O(file size). The report and rejected-line lists still require memory proportional to the output and rejected records. For a 50,000-line file, line-by-line reading is therefore preferable when input size may grow.

TASK 2 – FROM REQUIREMENT TO FILE-PROCESSING DESIGN (8 MARKS)

Task 2.1 – Model the data (2 marks)

Input format: One record per line in each daily UTF-8 text file. Example: `192311001,2026-10-08,P`. The fields are register number, date in YYYY-MM-DD format and status P or A.

Data structures: Use a dictionary keyed by register number. Each value stores present and absent counts, for example `{'192311001': {'P': 18, 'A': 2}}`. Use a list for damaged-record messages.

Output files: attendance_summary.csv contains register number, present count, absent count and percentage. below_75.csv contains students whose percentage is below 75. damaged_lines.log contains source file name, line number, original line and reason.

Task 2.2 – File operations (2 marks)

Operation	Choice and reason
Read mode	Use r to read daily attendance files without changing them.
Write mode	Use w to create fresh monthly reports, replacing old reports for a new run.
Append mode	Use a for an ongoing error log if errors must be retained across runs; otherwise create a new log with w for each monthly run.
Encoding	Use encoding='utf-8' to read common text files consistently.
Methods	Use pathlib.Path.glob() to list daily files, file iteration to process one line at a time, and write() to write report rows.

Task 2.3 – Monthly processing pseudocode (3 marks)

```
ALGORITHM: GENERATE_MONTHLY_ATTENDANCE
INPUT: folder_path containing daily attendance files
OUTPUT: attendance_summary.csv, below_75.csv, damaged_lines.log

1. CREATE empty dictionary counts
2. CREATE empty list damaged
3. GET all matching daily files using pathlib folder_path.glob("*.txt")
4. FOR EACH file_path in daily files
      TRY
          OPEN file_path in read mode with UTF-8 encoding
          FOR EACH line with line_number starting at 1
              REMOVE surrounding whitespace from line
              IF line is empty, CONTINUE
              SPLIT line by comma into fields
              IF field count is not 3
                  ADD file name, line number, line and reason to damaged
                  CONTINUE
              ASSIGN register_no, date_text, status
              IF date is invalid OR status is not P or A
                  ADD damaged record to damaged
                  CONTINUE
              IF register_no is not in counts
                  CREATE counts[register_no] with P=0 and A=0
              INCREMENT the matching P or A count
      CATCH FileNotFoundError, PermissionError, UnicodeDecodeError
          ADD file-level error to damaged
5. OPEN attendance_summary.csv in write mode
6. WRITE header
7. FOR EACH student in counts
      total = P count + A count
      IF total > 0, percentage = P count / total * 100
      WRITE student, counts and percentage
      IF percentage < 75, WRITE student to below_75.csv
8. WRITE every damaged entry to damaged_lines.log
9. END
```

Implementation note: if the month has a fixed number of instructional days, use that denominator instead of P + A. The assignment does not specify holidays or expected working days, so this design uses the number of valid attendance records received for each student.

Task 2.4 – Trace (1 mark)

Input file day01.txt:

```
192311001,2026-10-01,P
192311002,2026-10-01,A
192311003,2026-10-01
192311001,2026-10-01,A
```

Line	Processing	Result
1	3 fields; status P	192311001: P=1, A=0
2	3 fields; status A	192311002: P=0, A=1
3	Only 2 fields	Damaged log: day01.txt, line 3, expected 3 fields
4	3 fields; status A	192311001: P=1, A=1

After this file, 192311001 has 50% attendance and 192311002 has 0% attendance. Both are below 75%. Student 192311003 is not counted because the record is damaged.

Alternative concept and complexity analysis

Re-reading all f files for every one of s students means scanning approximately s × f × l lines, so time is O(s·f·l). The dictionary approach reads each line once and updates a count, so time is O(f·l) on average. The dictionary uses O(s) space for s students, plus O(e) for e logged errors. The repeated-scan approach can use O(1) extra counting space per student if reports are written immediately, but it performs much more input work.

TASK 3 – DESIGN THE ERROR HANDLING (9 MARKS)

Task 3.1 – Identify failures (3 marks)

Failure	Likely built-in exception / handling
Folder does not exist	FileNotFoundError when attempting to open a path inside the missing folder; check folder existence with pathlib.
Daily file does not exist	FileNotFoundError
No permission to read file	PermissionError
Invalid UTF-8 byte sequence	UnicodeDecodeError
Wrong number of fields	Custom InvalidRecord; alternatively ValueError after validation
Missing field access	IndexError if code accesses a missing list element without checking length
Invalid date or status	ValueError from date parsing or custom InvalidRecord for invalid status
Empty file	No exception; iteration returns no lines. Detect with a record counter and report 'empty file' if required.
Disk full / write failure	OSError, possibly a more specific subclass such as IsADirectoryError or PermissionError
Wrong variable name	NameError

Task 3.2 – Custom exception (2 marks)

```
class InvalidRecord(Exception):
    """Raised when an attendance row is malformed."""

def parse_line(line):
    parts = line.strip().split(",")
    if len(parts) != 3:
        raise InvalidRecord(
            f"expected 3 fields, found {len(parts)}"
        )
    register_no, date_text, status = parts
    if status not in ("P", "A"):
        raise InvalidRecord(f"invalid status '{status}'")
    return register_no, date_text, status
```

parse_line() raises InvalidRecord because it is responsible for validating one record. summarise() catches InvalidRecord because it can reject one bad row and continue processing the remaining rows.

Task 3.3 – Complete try statement and completed code (3 marks)

```
import os

class InvalidRecord(Exception):
    """Raised when a line is not in the expected format."""

def parse_line(line):
    parts = line.strip().split(",")
    if len(parts) != 3:
        raise InvalidRecord(
            f"expected 3 fields, found {len(parts)}"
        )
    register_no, date_text, status = parts
    if status not in ("P", "A"):
        raise InvalidRecord(f"invalid status '{status}'")
    return register_no, date_text, status

def summarise(path):
    present, absent, errors = 0, 0, []
    try:
        with open(path, "r", encoding="utf-8") as file:
            for line_number, line in enumerate(file, start=1):
                try:
                    register_no, date_text, status = parse_line(line)
                    if status == "P":
                        present += 1
                    else:
                        absent += 1
                except InvalidRecord as error:
                    errors.append((line_number, str(error)))
    except FileNotFoundError as error:
```

```
            raise RuntimeError(f"Attendance file not found: {path}") from error
        except PermissionError as error:
            raise RuntimeError(f"Cannot read attendance file: {path}") from error
        except UnicodeDecodeError as error:
            raise RuntimeError(f"Invalid text encoding in: {path}") from error
        else:
            if present + absent == 0 and not errors:
                print("Warning: file is empty or has no records")
        finally:
            print(f"Finished processing: {path}")

        return present, absent, errors

    if __name__ == "__main__":
        import tempfile
        data = (
            "192311001,2026-10-08,P\n"
            "192311002,2026-10-08,A\n"
            "192311003,2026-10-08\n"
            "192311004,2026-10-08,L\n"
        )
        with tempfile.NamedTemporaryFile(
            "w", suffix=".csv", delete=False, encoding="utf-8"
        ) as tmp:
            tmp.write(data)
            temp_name = tmp.name

        print(summarise(temp_name))
        os.remove(temp_name)
```

Expected data result: `(1, 1, [(3, 'expected 3 fields, found 2'), (4, "invalid status 'L'")])`. The program also prints a completion message from finally before printing the returned tuple.

To test a missing file, call summarise('missing_attendance.csv') inside a try block and catch RuntimeError. The RuntimeError is raised from the original FileNotFoundError, so the original cause remains available through exception chaining.

```
try:
    print(summarise("missing_attendance.csv"))
except RuntimeError as error:
    print(error)
    print("Original cause:", type(error.__cause__).__name__)
```

Task 3.4 – Misuse experiment (1 mark)

```
# Deliberately incorrect:
try:
    total_present = presnt_count + 1  # misspelled variable
except Exception:
    pass
```

The misspelled name raises NameError. The broad except Exception: pass catches it and does nothing, so the program continues without showing the error. The result may be incomplete or incorrect, and the programmer cannot identify the faulty variable. Specific exception handling and logging are safer.

Alternative concept and complexity analysis

With exception-based validation, the program attempts to parse a row and catches InvalidRecord when parsing fails. With look-before-you-leap validation, it first checks the number of fields and status using if statements, then processes valid values. Both are O(1) work per line and O(n) for n lines. When bad lines are rare, exception handling is readable and the exceptional path is uncommon. When bad lines are common, repeated exception creation and traceback handling adds overhead; explicit if checks are generally faster and clearer. Both use O(1) working space per line, excluding stored error messages.

TASK 4 – FILE AND EXCEPTION FORENSICS (7 MARKS)

Task 4.1 – Locate the defects (2 marks)

• Earlier days are lost because save_returns() opens returns.log in w mode. Write mode truncates the file every time the function is called.

• The book names are joined because write(b) does not add a newline. Each book entry is written directly after the previous entry.

• If write() fails, log.close() is skipped because it is not protected by a with statement or finally block. The file may remain open until Python or the operating system closes it.

• Only FileNotFoundError is caught. Other I/O errors are not recorded, and print('missing') is not a persistent error log.

• open(day_file).read() also leaves file closure to cleanup and reads the whole day file into memory.

Task 4.2 – Construct evidence (2 marks)

```
# Demonstration of the original defect
def save_returns_bad(books):
    log = open("returns_bad.log", "w")
    for book in books:
        log.write(book)
    log.close()

def process_bad(day_file):
    try:
        books = open(day_file).read().split("\\n")
        save_returns_bad(books)
    except FileNotFoundError:
        print("missing")

# Create three sample day files, then process them.
for filename, content in [
    ("day1.txt", "Book-A\\nBook-B"),
    ("day2.txt", "Book-C"),
    ("day3.txt", "Book-D\\nBook-E")
]:
    with open(filename, "w", encoding="utf-8") as f:
        f.write(content)

for filename in ("day1.txt", "day2.txt", "day3.txt"):
    process_bad(filename)

with open("returns_bad.log", "r", encoding="utf-8") as f:
    print(f.read())
# Output: Book-DBook-E
# Only day 3 remains, and the two entries are joined.
```

Task 4.3 – Repaired program (2 marks)

```
def save_returns(books, log_name="returns.log"):
    with open(log_name, "a", encoding="utf-8") as log:
        for book in books:
            if book.strip():
                log.write(book.strip() + "\\n")

def process(day_file):
    try:
        with open(day_file, "r", encoding="utf-8") as source:
            books = [line.strip() for line in source if line.strip()]
        save_returns(books)
    except (FileNotFoundError, PermissionError, UnicodeDecodeError, OSError) as error:
        with open("errors.log", "a", encoding="utf-8") as error_file:
            error_file.write(f"{day_file}: {type(error).__name__}: {error}\\n")

if __name__ == "__main__":
    for filename in ("day1.txt", "day2.txt", "day3.txt"):
        process(filename)
```

Append mode preserves earlier days. Each return is written with a newline. Both source and destination files use with statements. File errors are appended to errors.log.

Task 4.4 – Prevent recurrence (1 mark)

• Use with open(...) so files are reliably closed even when an exception occurs.

• Choose file mode deliberately: w replaces a file, while a preserves earlier contents; write a newline between records.

Alternative concept and complexity analysis

Manual handling can use try/finally: open the file, perform the operation inside try, and call close() in finally after checking that the file was opened. The with statement uses the context manager protocol to perform the same cleanup more safely. Neither approach changes the O(n) time required to write n characters or the O(1) additional working space for streaming writes. With a single file open at a time, each version normally has at most one log file open at that point; if the source is also open while the log is opened, two files may be open simultaneously. A missing close in the manual version can leave handles open after errors, whereas with closes them automatically.

TASK 5 – THE SAME UTILITY ORGANISED FOUR WAYS (7 MARKS)

Task 5.1 – Four organisations (3 marks)

(a) One script: Put reading, statistics and report functions in attendance_all.py. Keep execution code under if __name__ == '__main__'.

```
# attendance_all.py
def read_records(path):
    records = []
    with open(path, encoding="utf-8") as file:
        for line in file:
            parts = line.strip().split(",")
            if len(parts) == 3 and parts[2] in ("P", "A"):
                records.append((parts[0], parts[2]))
    return records

def percentage(present, absent):
    total = present + absent
    return (present / total * 100) if total else 0.0

def write_report(rows, path):
    with open(path, "w", encoding="utf-8") as file:
        file.write("Register,Percentage\\n")
        for register, value in rows:
            file.write(f"{register},{value:.2f}\\n")

if __name__ == "__main__":
    records = read_records("attendance.txt")
    counts = {}
    for register, status in records:
        counts.setdefault(register, [0, 0])
        counts[register][0 if status == "P" else 1] += 1
    rows = [(r, percentage(v[0], v[1])) for r, v in counts.items()]
    write_report(rows, "summary.csv")
```

(b) One utility module imported by a main script:

```
# attendance_utils.py
def percentage(present, absent):
    total = present + absent
    return present / total * 100 if total else 0.0

# main.py
from attendance_utils import percentage

if __name__ == "__main__":
    print(percentage(18, 2))
```

(c) Three modules in one folder:

```
# attendance_tools/reader.py
def read_records(path):
    records = []
    with open(path, encoding="utf-8") as file:
        for line in file:
            parts = line.strip().split(",")
            if len(parts) == 3 and parts[2] in ("P", "A"):
                records.append((parts[0], parts[2]))
    return records

# attendance_tools/stats.py
def percentage(present, absent):
    total = present + absent
    return present / total * 100 if total else 0.0

# attendance_tools/report.py
def write_report(rows, path):
    with open(path, "w", encoding="utf-8") as file:
        file.write("Register,Percentage\\n")
        for register, value in rows:
            file.write(f"{register},{value:.2f}\\n")

# main.py
from attendance_tools.reader import read_records
from attendance_tools.stats import percentage
from attendance_tools.report import write_report
```

(d) Package attendance/ with __init__.py and submodules:

```
attendance/
    __init__.py
    reader.py
    stats.py
    report.py
main.py

# attendance/__init__.py
from .reader import read_records
from .stats import percentage
from .report import write_report

# attendance/reader.py
def read_records(path):
    records = []
    with open(path, encoding="utf-8") as file:
        for line in file:
            parts = line.strip().split(",")
            if len(parts) == 3 and parts[2] in ("P", "A"):
                records.append((parts[0], parts[2]))
    return records

# attendance/stats.py
def percentage(present, absent):
    total = present + absent
    return present / total * 100 if total else 0.0

# attendance/report.py
def write_report(rows, path):
    with open(path, "w", encoding="utf-8") as file:
        file.write("Register,Percentage\\n")
        for register, value in rows:
            file.write(f"{register},{value:.2f}\\n")

# main.py
from attendance import read_records, percentage, write_report

if __name__ == "__main__":
    print(percentage(18, 2))
```

The special file name is **__init__.py** (two underscores before and after init). It marks the folder as a regular package and can expose selected functions.

Task 5.2 – Change the requirement (2 marks)

To add a report for students absent for three consecutive days, the reader must preserve each student's date-wise status. The statistics logic must detect a run of three consecutive A records. The report module must write the new list to a separate file.

Organisation	Files to change	Reason
One script	1	Add the detection and output code in the same file.
One utility module + main	2	Change the utility module and main script to call the new function.
Three modules	2 or 3	Add the function in stats.py and update report.py; main.py must call both if it controls the workflow.
Package	2 or 3	Add detection in stats.py and output in report.py; update __init__.py if exporting the new function, and main

Task 5.3 – Compare organisations (2 marks)

Organisation	Readability / reuse / testing / naming
One script	Easy for a very small program, but becomes long. Functions are less convenient to reuse independently. Testing is possible but the file h
One module + main	Separates reusable logic from execution. Easier to test utility functions. Useful for a small project, but the module can become crowded.
Three modules	Clear separation of reading, statistics and reporting. Each module can be tested independently and reused. Generic module names can
Package	Best structure for a growing project. Related modules share a namespace and can be extended. Easier to maintain and test, but requires

Import differences: `import attendance.stats` imports the submodule and it is normally accessed as `attendance.stats.percentage(...)`. `from attendance import stats` imports the stats submodule into

the current namespace, so use `stats.percentage(...)`. `from attendance.stats import percentage` imports the function directly, so call `percentage(...)`. Direct imports are concise but can create naming conflicts.

Alternative concept and complexity analysis

If all functions remain in one script, the program can still work by defining each function and calling them in the correct order. A package adds an organisational structure, not a different attendance algorithm. Python loads an imported module once per interpreter process and caches it in sys.modules; later imports normally reuse the cached module. Importing has a one-time startup cost related to module loading and execution. Package memory is proportional to the imported module code and objects, generally small for this utility. Finding and changing a function is easiest in a small one-script project, but becomes harder as the file grows. In a package, the function's responsibility and location are easier to identify.

TASK 6 – DESIGN CHALLENGE: COLLEGE FEST REGISTRATION DESK (13 MARKS)

Stage 1 – Interpret the problem (2 marks)

Inputs: Five department text files. Each non-empty line has register number, name, department, e-mail and event list separated by commas. Event names are separated by semicolons.

Outputs: clean_master.csv, one participant file per event, summary.txt and errors.log.

Rules: A registration must have exactly five fields. Register number must not be empty. E-mail must contain one @, a non-empty local part, a domain containing a dot, and no spaces. Event list must contain at least one non-empty event. Duplicate register numbers are kept only once. A damaged line, invalid e-mail or missing input file is logged; processing continues.

• All existing department files are processed even if one file is missing.

• Each accepted register number appears only once in the master list.

• Every accepted event is included in its event participant file.

• Invalid records are written to the error log with file name and line number.

• The summary reports accepted unique registrations, duplicates, rejected records and event totals.

Stage 2 – Processing flow (2 marks)

```
START
  |
Find the five department files
  |
For each department file
  |
Open file? ---- no ----> Log missing-file error ----> Next file
  |
Read one line
  |
Is line empty? -- yes --> Next line
  |
Split into five fields
  |
Validate fields and e-mail
  |                     |
Invalid               Valid
  |                     |
Log error             Register number already seen?
  |                     |                 |
Next line             Yes               No
                        |                 |
                  Log duplicate     Add register to seen set
                                          |
                                    Add to clean master
                                          |
                                    Add student to each event
                                          |
                                       Next line
  |
Write master, event files, summary and error log
  |
END
```

Stage 3 – Critical design decisions (2 marks)

• **Duplicate detection:** Keep a set named seen_registers. Before accepting a row, test whether its register number is already in the set. Set membership is O(1) on average.

• **E-mail validation:** Use string methods: strip(), count('@'), split('@'), check for whitespace, and check that the domain contains a dot and does not begin or end with a dot. This is basic validation, not a guarantee that the mailbox exists.

• **Exceptions:** Catch FileNotFoundError and PermissionError around opening each department file. Catch UnicodeDecodeError around reading. Use a custom InvalidRegistration exception for damaged records and handle it

inside the line loop so later lines continue.

• **Duplicate policy:** First valid registration for a register number is retained. Later duplicates are logged and ignored.

Stage 4 – Package design (4 marks)

Suggested structure:

```
fest_registration/
    __init__.py
    reader.py       # read department files
    validation.py   # validate rows and e-mail
    processor.py    # remove duplicates and group events
    writer.py       # write output files
main.py
input/
    arts.txt
    science.txt
    commerce.txt
    engineering.txt
    management.txt
output/
```

Module responsibilities:

Module	Functions
validation.py	InvalidRegistration; validate_email(email); parse_record(line)
reader.py	read_department_file(path, error_list)
processor.py	process_files(paths) – validate, de-duplicate and group event participants
writer.py	write_master(rows); write_event_files(events); write_summary(stats); write_errors(errors)
main.py	Build input paths, call processor and writer functions.

Pseudocode:

```
ALGORITHM: PROCESS_FEST_REGISTRATIONS
INPUT: list of five department file paths
OUTPUT: clean master, event participant files, summary and error log

1. CREATE empty set seen_registers
2. CREATE empty list master, errors
3. CREATE empty dictionary event_groups
4. SET duplicate_count = 0, rejected_count = 0
5. FOR EACH input_path
      TRY OPEN input_path using UTF-8
      IF file cannot be opened
          RECORD file error; CONTINUE to next input_path
      FOR EACH line and line_number
          IF line is blank, CONTINUE
          TRY
              SPLIT line by comma
              REQUIRE exactly five fields
              STRIP each field
              VALIDATE register number, name, department
              VALIDATE e-mail using string methods
              SPLIT events by semicolon; remove empty event names
              IF register number is in seen_registers
                  INCREMENT duplicate_count
                  RECORD duplicate; CONTINUE
              ADD register number to seen_registers
              ADD registration to master
              FOR EACH event
                  ADD registration to event_groups[event]
          CATCH InvalidRegistration
              INCREMENT rejected_count
              RECORD input_path, line_number and reason
5. WRITE master list
6. FOR EACH event in event_groups
      WRITE a separate participant file
7. WRITE summary counts and event totals
8. WRITE errors list
9. END
```

Stage 5 – Implementation and demonstration (3 marks)

The following is a complete, compact implementation that can be saved as **main.py**. It uses separate functions so it can later be moved into the package structure shown above.

```
from pathlib import Path
import csv

class InvalidRegistration(Exception):
    pass

def validate_email(email):
    email = email.strip()
    if " " in email or email.count("@") != 1:
        return False
    local, domain = email.split("@")
    return bool(local and domain and "." in domain
                and not domain.startswith(".")
                and not domain.endswith("."))

def parse_record(line):
    fields = [part.strip() for part in line.strip().split(",")]
    if len(fields) != 5:
        raise InvalidRegistration(
            f"expected 5 fields, found {len(fields)}"
        )
    reg, name, dept, email, event_text = fields
    if not reg or not name or not dept:
        raise InvalidRegistration("register, name and department are required")
    if not validate_email(email):
        raise InvalidRegistration(f"invalid email '{email}'")
    events = [e.strip() for e in event_text.split(";") if e.strip()]
    if not events:
        raise InvalidRegistration("at least one event is required")
    return {
        "register": reg, "name": name, "department": dept,
        "email": email, "events": events
    }

def process_files(paths):
    seen = set()
    master = []
    event_groups = {}
    errors = []
    duplicates = 0
    rejected = 0

    for path in paths:
        try:
            with open(path, "r", encoding="utf-8") as file:
                for line_no, line in enumerate(file, start=1):
                    if not line.strip():
                        continue
                    try:
                        record = parse_record(line)
                        reg = record["register"]
                        if reg in seen:
                            duplicates += 1
                            errors.append(
                                f"{path}:{line_no}: duplicate register {reg}"
                            )
                            continue
                        seen.add(reg)
                        master.append(record)
                        for event in record["events"]:
                            event_groups.setdefault(event, []).append(record)
                    except InvalidRegistration as error:
                        rejected += 1
                        errors.append(f"{path}:{line_no}: {error}")
        except (FileNotFoundError, PermissionError, UnicodeDecodeError, OSError) as error:
            errors.append(f"{path}: {type(error).__name__}: {error}")

    return master, event_groups, errors, duplicates, rejected

def write_outputs(master, event_groups, errors, duplicates, rejected):
    out = Path("output")
    out.mkdir(exist_ok=True)

    master_path = out / "clean_master.csv"
    with open(master_path, "w", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        writer.writerow(["Register", "Name", "Department", "Email", "Events"])
```

```
        for r in master:
            writer.writerow([r["register"], r["name"], r["department"],
                             r["email"], ";".join(r["events"])])

    for event, participants in event_groups.items():
        safe_name = "".join(c if c.isalnum() else "_" for c in event)
        with open(out / f"{safe_name}.csv", "w", newline="",
                  encoding="utf-8") as file:
            writer = csv.writer(file)
            writer.writerow(["Register", "Name", "Department", "Email"])
            for r in participants:
                writer.writerow([r["register"], r["name"],
                                 r["department"], r["email"]])

    with open(out / "summary.txt", "w", encoding="utf-8") as file:
        file.write(f"Unique registrations: {len(master)}\\n")
        file.write(f"Duplicates ignored: {duplicates}\\n")
        file.write(f"Rejected records: {rejected}\\n")
        file.write("Event totals:\\n")
        for event, participants in sorted(event_groups.items()):
            file.write(f"{event}: {len(participants)}\\n")

    with open(out / "errors.log", "w", encoding="utf-8") as file:
        for error in errors:
            file.write(error + "\\n")

if __name__ == "__main__":
    input_dir = Path("input")
    departments = [
        input_dir / "arts.txt",
        input_dir / "science.txt",
        input_dir / "commerce.txt",
        input_dir / "engineering.txt",
        input_dir / "management.txt"
    ]
    result = process_files(departments)
    write_outputs(*result)
    print("Processing completed. Check the output folder.")
```

Sample test data: Save the following as input/arts.txt. The second row is a duplicate register number, the third has an invalid e-mail and the fourth is damaged.

```
192311001,Asha,Arts,asha@example.com,Dance;Singing
192311002,Bala,Arts,bala@example.com,Drama
192311001,Asha,Arts,asha@example.com,Dance
192311003,Charan,Arts,charan.example.com,Music
192311004,Divya,Arts,divya@example.com
```

Also create input/science.txt with a valid registration, and leave input/commerce.txt absent to test missing-file handling. Create the remaining two files with valid rows. Run the program from the project folder using `python main.py`.

Expected demonstration: the program completes without stopping. Unique valid register numbers are written once to clean_master.csv. Dance, Singing and Drama files are created as applicable. The duplicate, invalid e-mail, damaged line and missing commerce.txt file are recorded in errors.log. Exact counts depend on the other sample rows you add.

Alternative concept and complexity analysis

If duplicates are checked by searching a list of all earlier register numbers, each new registration may require scanning up to n previous values. Total comparisons can grow as 1 + 2 + … + (n–1), giving $O(n^2)$ time. With a set, membership and insertion are O(1) on average, so processing n registrations is O(n) on average. Both approaches store accepted register numbers, so extra space is O(n). The set approach is faster for large registration files.

APPENDIX – README TEMPLATE AND EXECUTION CHECKLIST

README.md template

```
# CSA08 - Assignment 4 (CO4)

Name: YOUR NAME
Register Number: YOUR REGISTER NUMBER
Python Version: Python 3.x

## Libraries
- Python standard library only (csv, pathlib, os, decimal, tempfile)

## Run commands
- Task 1: python task1.py
- Task 2: python task2.py
- Task 3: python task3.py
- Task 4: python task4.py
- Task 5: python main.py
- Task 6: python main.py

## Results
- Task 1: Mess bill file processed; rejected records listed.
- Task 2: Monthly attendance summary and damaged-line log generated.
- Task 3: Valid records counted; invalid rows handled.
- Task 4: Return log preserves all days; errors saved.
- Task 5: Attendance utility organised using modules/package.
- Task 6: Fest master list, event lists, summary and error log generated.

## Requirements
The project uses only the Python standard library.
```

Execution and GitHub checklist

• Create the repository named CSA08-RegisterNumber-Assignment4.

• Create Task01 through Task06 folders.

• Save each program as a .py file in the correct folder.

• Create sample input files and run every program using Python 3.

• Check the output against the expected results in this report.

• Capture screenshots showing the code execution and generated output files.

• Add screenshots and input/output files to the matching task folders.

• Add README.md and requirements.txt. If no external libraries are used, state that only the Python standard library is required.

• Commit each task separately; do not upload everything in one final commit.

• Check repository visibility or add the faculty as a collaborator.

• Paste the working GitHub URL on the cover page and in the submission field.

Viva preparation – quick questions

Question	Short answer
Why use with open()?	It closes the file automatically, including when an exception occurs.
Difference between w and a?	w overwrites; a adds to the end.
Why use encoding='utf-8'?	It gives consistent text decoding and helps identify invalid byte sequences.
What is a custom exception?	A user-defined exception for a specific application error.
Why avoid except Exception: pass?	It hides errors and makes debugging difficult.
What is a module?	A Python file containing reusable code.
What is a package?	A folder that groups related Python modules.

Question	Short answer
Why use a set for duplicates?	Membership checking is O(1) on average.
What is Big-O?	A way to describe how time or memory grows as input size increases.

Final reminder: This is a complete written solution guide, but you must personally run and verify the code, take genuine screenshots, fill in your details and submit your own working GitHub repository link.